COMPILER DESIGN PROJECT

Error Buddy;

The Compiler That Explains Its Own Errors

A beginner compiler in C++17 that reports every mistake on the line where the fix belongs, in plain words, with one concrete fix.

5 PHASES · 3 SOURCE FILES | C++17 | 24 / 24 ON THE FIX LINE

PROGRAM

```
int a = 5;
int b = 10
print a + b;
```

× ERROR | LINE 2 · SYNTAX

```
Expected ';' at the end of the statement.
Suggestion: Add a semicolon after '10'.
```


TEAM

Vardan Pandya
24BCE2608

Kunal Sharma
24BCE2641

PROBLEM STATEMENT

Compilers tell beginners that something is wrong. Not where to fix it, or how.

A student forgets one semicolon. The compiler points at the next line, says *"expected ',' or ';' before 'printf'"* and stops. The message is accurate for someone who knows the grammar. For a beginner it names the wrong line, uses parser vocabulary and never says what to type. Some real mistakes, like dividing by a variable that is zero, get no message at all.

- ✕ **Wrong line**
- ✕ **Grammar words**
- ✕ **No fix suggested**
- ✕ **Some mistakes not reported**

RESEARCH QUESTION

How can a compiler report each beginner mistake on the line where the fix belongs, in plain words, with one concrete fix?

THE PROBLEM

Real mistakes. Unhelpful messages.

Three beginner programs (C equivalents), compiled with gcc 11.4 -Wall -Wextra. The first diagnostic is shown.

A missing semicolon

```
1  int a = 5;
2  int b = 10
3
4  // show the total
5  printf("%d\n", a + b);
```

gcc: line 5: expected ',' or ';' before 'printf'

WRONG LINE · FIX IS ON 2

Two operators in a row

```
1  int x = 4;
2  int y = x + * 3;
3  printf("%d\n", y);
```

gcc: invalid type argument of unary '*' (have 'int')

GRAMMAR WORDS, NO FIX

Dividing by a variable that is 0

```
1  int a = 10;
2  int zero = a - 10;
3  int b = a / zero;
4  printf("%d\n", b);
```

gcc: no error and no warning. The program crashes or misbehaves when it runs.

NOT REPORTED

20 of 24 on the right line. 1 not reported at all.

We ran 24 classic beginner mistakes through gcc 11.4 and ErrorBuddy (slide 17).

THE PROBLEM

Where does a compiler report a mistake?

Every phase checks one thing. The report goes where the parser noticed the problem, not where the student must edit.

1 Lexical	**2 Syntax**	**3 Semantic**	**4 Code**	**5 Explain?**
are the words valid?	does the grammar fit?	do the names exist?	generate output	where is the fix, and what is it?

THE DETECTION POINT

A parser only knows a `;` is missing when it reads the **next** token. If that token is on line 3, line 3 is reported, although the fix belongs at the end of line 2. When the next statement is after blank lines or comments, the report moves even further away (gcc reported line 5 for a fix on line 2).

```
2  int b = 10        <- fix belongs here
3
4  // show the total
5  printf(...)       <- gcc reports here
```

THE ANALOGY

A teacher marking an essay circles the word that is misspelled and writes the right spelling in the margin. A compiler that reports the next line and says "expected ... before" is like a teacher circling the next word on the page and writing "unexpected word".

Beginners need the **first** kind of feedback: the place, the rule and the fix.

THE IDEA

Introducing ErrorBuddy

A small compiler whose main output, when something is wrong, is a lesson.

THE THESIS

A compiler should not just detect a mistake. It should explain it: the line, the rule and the fix.

Every error is a structured object with four fields, built by the phase that found it, and printed in one fixed format.

`LINE` `PHASE` `MESSAGE` `SUGGESTION`

1. **Report on the fix line**
   A missing ';' is reported on the line of the token before it.
2. **Name the rule in plain English**
   "Two operators in a row: '+' followed by '*'."
3. **Give one concrete fix**
   "Add a semicolon after '10'." Not a grammar lesson.
4. **Say which phase found it**
   Lexical, Syntax or Semantic Analysis.
5. **Stop before generating code**
   No partial three-address code is printed for a broken program.

THE IDEA

What makes it different

Ordinary compilers stop at detection. ErrorBuddy adds a diagnosis step before every error.

01

Four-field errors

CompileError { line, phase, message, suggestion } is the only way any phase reports a problem.

02

Diagnosis from nearby tokens

Before reporting, the parser looks at the previous and next token to name the likely cause: extra ')', missing operator, extra '='.

03

Measured typo hints

"Did you mean 'score'?" uses Levenshtein distance with a limit chosen by testing four policies on 23 typo cases.

04

Compile-time values

The symbol table tracks each variable's value, so division by a variable that is 0 is caught before the program runs.

THE IDEA

The language at a glance

A small C-like language, kept small on purpose so that every phase is complete.

EXAMPLE PROGRAM · examples/valid.txt

```
1  // A valid ErrorBuddy program
2  int age = 20;
3  int score = 10;
4  score = score + age * 2;
5  int bonus = (score - 5) / 5;
6  print score;
7  print bonus * (age + 1);
```

OUTPUT **12** TAC instructions · **3** variables · exit 0

TYPES	`int` only; every variable needs a starting value
STATEMENTS	`int x = e;  x = e;  print e;`
OPERATORS	`+ - * /` with precedence, and `( )`
COMMENTS	`// to the end of the line`
RULES	every statement ends with `;` · integer division
NOT INCLUDED	unary minus, if/else, loops, functions, input
ON ERROR	stops at the first error; no TAC is printed

ARCHITECTURE

The architecture: five phases

Three C++ files. Phases 2 to 4 run together in one pass of the parser (syntax-directed translation).

1

Lexical analysis

`lexer.cpp`

characters → tokens with line numbers

`OUT · vector<Token>`

→

`parser.cpp · ONE PASS`

2

Syntax

recursive descent

3

Semantic

symbol table checks

4

TAC

t1 = a op b

→

D

Driver

`main.cpp`

prints TAC, symbol table and phase summary

`EXIT 0`

5

Error reporting · CompileError { line, phase, message, suggestion }

Defined in lexer.h. Any phase throws it at the first problem; main.cpp catches it, prints it and exits with code 1. No TAC is shown.

■ phase □ shared error channel exit codes: 0 ok · 1 compile error · 2 usage · 3 internal

PHASES

Phases 1 to 3 in detail

From raw text to checked statements.

1 Lexical analysis

`lexer.cpp · tokenize()`

- One loop over the characters
- Counts newlines, so every token has a line
- Skips spaces and // comments
- `int`, `print` only as whole words
- Rejects `@`, `2nd`, numbers above INT_MAX

IN	source string
OUT	tokens ending in END_OF_FILE

2 Syntax analysis

`parser.cpp · statement() …`

- Recursive descent: one function per rule
- `expression → term → factor` gives precedence
- While loops give left associativity
- Nesting limit of 200 for (
- Diagnoses a missing ';' before reporting

IN	tokens
OUT	recognised statements

3 Semantic analysis

`parser.cpp · symbolTable`

- `map<name, {line, value}>`
- Undeclared names, with 'Did you mean'
- Duplicate declarations
- Use inside its own declaration
- Division by zero and int overflow

IN	each name and operator
OUT	checked operands

PHASES

Phases 4 and 5 in detail

Generating code when the program is right, explaining it when it is not.

4 Three-address code

`parser.cpp · binary(), newTemp()`

- At most one operator per instruction
- Temporaries t1, t2… skip names the user wrote
- Parentheses add grouping, not instructions
- Emitted as each statement is parsed
- Printed only if every phase passes

IN	operands and operators
OUT	TAC list in source order

5 Error reporting

`lexer.h · CompileError`

- Thrown by any phase at the first problem
- `expectSemicolon()`, `factorError()`, `unknownStatementError()` diagnose the cause
- Edit-distance typo hints
- Reported on the line where the fix belongs
- main.cpp prints it and exits with 1

IN	the failing token and its neighbours
OUT	line · phase · message · suggestion

ARCHITECTURE

Structure: files and shared parts

Every file has one job. Four data structures are shared between phases.

LAYER	FILE	JOB
Front end	`lexer.h / lexer.cpp`	Phase 1 and the shared CompileError
One pass	`parser.h / parser.cpp`	Phases 2–4 and the diagnosis helpers
Driver	`main.cpp`	Run phases, print result or error, exit code
Build & test	`Makefile`, `examples/`	make, make test (10 programs)
Front ends	`web/`, `site/`	Local playground, GitHub Pages site
Evaluation	`case_study/`	24 mistakes vs gcc, hint and lexer studies

SHARED COMPONENTS

Token

type, exact text and line number; made by the lexer, read by every later phase.

CompileError

line, phase, message, suggestion; the single error channel.

Symbol table

name → declaration line and current value.

TAC list

instructions in source order, printed by main.cpp.

ERRORS

What is a CompileError?

Four fields, filled by the phase that found the problem, printed in one fixed format.

```
struct CompileError {
    int line;
    std::string phase;
    std::string message;
    std::string suggestion;
};
```

ACTUAL OUTPUT · examples/missing_semicolon.txt

```
Error on line 2:
Expected ';' at the end of the statement.
Suggestion: Add a semicolon after '10'.

(Found during: Syntax Analysis)
Compilation stopped. No intermediate code was generated.
```

LINE

2

where the fix belongs, not where parsing stopped

PHASE

Syntax Analysis

which phase found it: Lexical, Syntax or Semantic

MESSAGE

Expected ';' at the end of the statement.

the rule that was broken, in plain English

SUGGESTION

Add a semicolon after '10'.

one concrete thing to type

ERRORS

How diagnosis works, and why it runs before the report

expectSemicolon() runs at the end of every statement. If the next token is not ';', it asks what it is instead.

1 **Statement parsed**
the next token should be ';'

2 **Look at it**
what is the token instead?

3 **Match a cause**
check the rules below, in order

4 **Pick the line**
where the fix belongs

5 **Throw**
CompileError with a fix

NEXT TOKEN	DIAGNOSIS	REPORTED ON
)	Found an extra ')' that has no matching '('.	that token's line
=	Unexpected '='. A statement can contain only one '='.	that token's line
name, number or (on the same line	Missing operator between the two values.	that token's line
anything else	Expected ';' at the end of the statement. Add a semicolon after the last token.	**the previous token's line**

The last rule is the fix-point rule.
It moved ErrorBuddy from 21 to 24 of 24 mistakes on the exact line (slide 18).

ERRORS

Five kinds of beginner mistake

The 24 mistakes in the case-study corpus, grouped by what went wrong.

1	**LEXICAL** Invalid characters and names	`M23 unknown symbol` `M24 name starts with a digit`	Lexical
2	**STRUCTURE** Missing ; and parentheses	`M01 missing ; (next line is prin` `M02 missing ; (blank line + comm` `M03 missing ; (next line is assi` `M04 missing ; on last statement` `M05 missing )` `M06 missing ) (expression split` `M07 extra )` `M12 empty parentheses`	Syntax
3	**EXPRESSIONS** Operators and missing values	`M08 two operators in a row` `M09 operator with no right opera` `M10 missing operator between val` `M11 missing value after =`	Syntax
4	**NAMES** Undeclared, misspelled, duplicate	`M13 undeclared variable` `M14 misspelled variable` `M15 misspelled print keyword` `M16 duplicate declaration` `M17 variable used in its own dec`	Syntax / Semantic
5	**VALUES** Division by zero and overflow	`M18 division by literal 0` `M19 division by a variable that` `M20 division by an expression th` `M21 arithmetic overflow` `M22 literal too large for int`	Lexical / Semantic

ERRORS

Typo hints: how close is close enough?

Levenshtein distance in a single rolling row. Four limits tested on 23 typo cases from case_study/corpus.py.

POLICY	RULE	RIGHT	WRONG	MISSED
k1	1 edit, all names	10	8	3
shipped	**1 edit if <=3 letters else 2, skip names <3**	**12**	**6**	**1**
k2	2 edits, all names	13	9	0
k3	3 edits, all names	13	10	0

EXAMPLES

```
scroe -> score   2 edits
prnt  -> print   1 edit
mx    -> max     1 edit
```

SHIPPED RULE

1 edit for names of up to 3 letters, 2 for longer names, never the word itself, and no hints for variables shorter than 3 letters. **Fewest wrong hints (6) with 12 right.**

PHASES

Meaning and code: the symbol table and TAC

No input and no loops means every value is known at compile time, so values are tracked as well as names.

SYMBOL TABLE AFTER valid.txt

NAME	DECLARED	VALUE
age	line 2	20
score	line 3	50
bonus	line 5	9

A name is added only after its starting value is parsed, so `int a = a + 1;` is caught.

THREE-ADDRESS CODE (12)

```
age = 20
score = 10
t1 = age * 2
t2 = score + t1
score = t2
t3 = score - 5
t4 = t3 / 5
bonus = t4
print score
t5 = age + 1
t6 = bonus * t5
print t6
```

CAUGHT BEFORE RUNNING

```
1  int a = 10;
2  int zero = a - 10;
3  int b = a / zero;
4  print b;
```

Error on line 3: Division by zero. Variable 'zero' is 0 at this point, so dividing by it is impossible.

GCC: NOT REPORTED

PROOF

Measured against GCC

Same mistakes written as C and compiled with gcc 11.4 -Wall -Wextra (first diagnostic). ErrorBuddy output is from the real compiler.

MISTAKE (FIX LINE IN CORPUS)	GCC 11.4	ERRORBUDDY
M01 missing ; (next line is print)	`line 3: expected ',' or ';' before 'printf'`	line 2: Expected ';' at the end of the statement.
M02 missing ; (blank line + comment after)	`line 5: expected ',' or ';' before 'printf'`	line 2: Expected ';' at the end of the statement.
M07 extra)	`line 2: expected ',' or ';' before ')' token`	line 2: Found an extra ')' that has no matching '('.
M08 two operators in a row	`line 2: invalid type argument of unary '*' (have 'int')`	line 2: Two operators in a row: '+' followed by '*'.
M10 missing operator between values	`line 3: expected ',' or ';' before 'b'`	line 3: Missing operator between 'a' and 'b'.
M14 misspelled variable	`line 3: 'scroe' undeclared (first use in this function); did you mean 'score'?`	line 3: Variable 'scroe' has not been declared.
M15 misspelled print keyword	`line 2: implicit declaration of function 'prinft'; did you mean 'printf'? [-Wimplicit-function-declaration]`	line 2: Unknown word 'prnt'.
M19 division by a variable that is 0	*nothing reported*	line 3: Division by zero. Variable 'zero' is 0 at this point, so dividing by it is impossible.

All 24 cases with both outputs are in the appendix and in case_study/out_placement.txt.

PROOF

Reported on the exact fix line

case_study/run_study.py: 24 mistakes, fix line labelled by hand before any compiler was run.

gcc 11.4



ErrorBuddy, detection point



ErrorBuddy, fix point (shipped)



Detection point = the line of the token where the parser noticed the problem. Fix point = the shipped rule.

24/24 mistakes caught by ErrorBuddy · gcc: 23

0 reports later than the fix line · gcc: 3

3 late gcc reports, all a missing ';'

PROOF

One program, end to end

examples/precedence.txt through every phase. b * 2 is computed before + because of the grammar.

1 · SOURCE

```
1  int a = 5;
2  int b = 10;
3  int c = a + b * 2;
4  print c;
```

2 · TOKENS (22 + EOF)

```
int c = a + b * 2 ;
KEYWORD IDENTIFIER ASSIGN
IDENTIFIER OPERATOR
IDENTIFIER
OPERATOR NUMBER SEMICOLON
```

3 · PARSE + CHECK

```
expression()
  term() -> a
  +
  term() -> b * 2
     binary() t1 = b * 2
  binary()  t2 = a + t1
```

4 · TAC

```
a = 5
b = 10
t1 = b * 2
t2 = a + t1
c = t2
print c
```

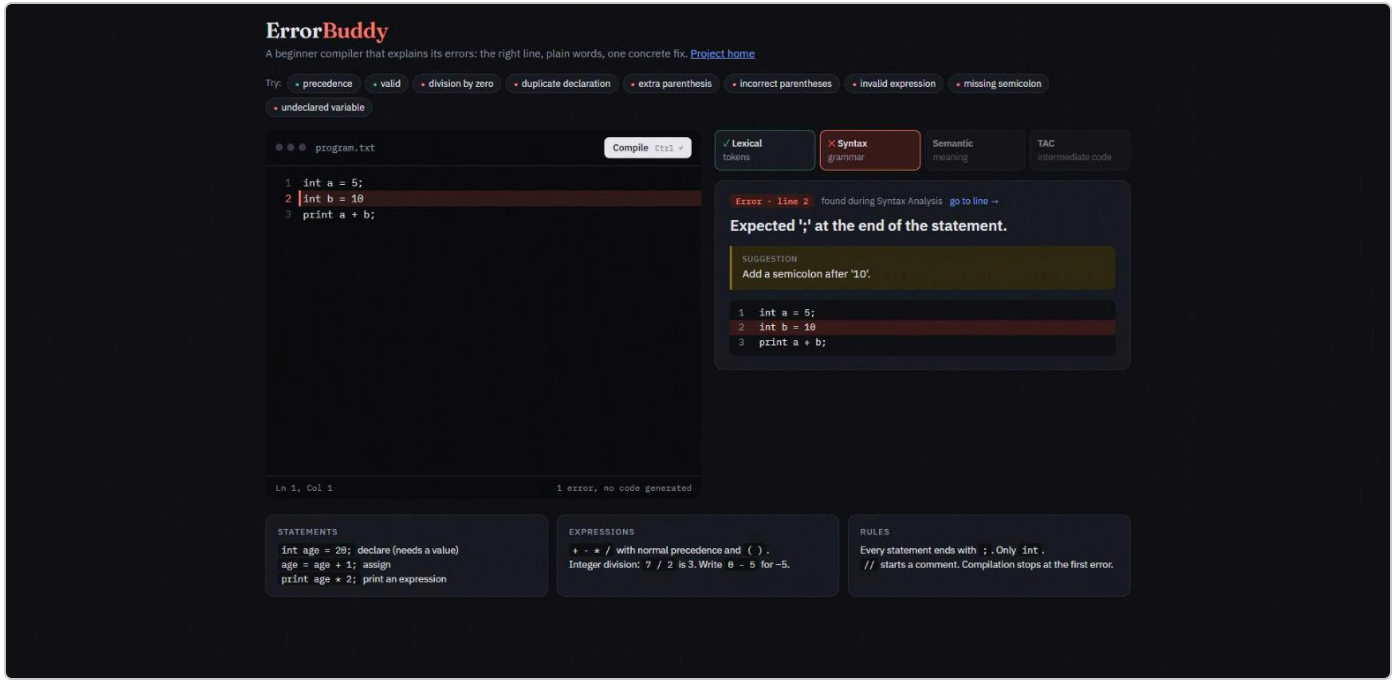
[OK] Lexical · [OK] Syntax · [OK] Semantic · [OK] Intermediate Code

Compilation successful · 6 instructions · exit 0

PROOF

The website and playground

Published on GitHub Pages with the documentation, this presentation and the final report.



Live site

`vardanp1.github.io/Compiler-Design-Project`

Playground

Every example shows the compiler's real output: error line highlighted, phase strip, TAC, symbol table and tokens.

Local playground

`make web` runs `web/server.py`, which calls the real binary on anything you type.

Source

`github.com/Vardanp1/Compiler-Design-Project`

PROOF

Testing: make test

Every example program has its expected output stored next to it. make test compares them byte for byte.

```
PASS  examples/division_by_zero.txt
PASS  examples/duplicate_declaration.txt
PASS  examples/empty.txt
PASS  examples/extra_parenthesis.txt
PASS  examples/incorrect_parentheses.txt
PASS  examples/invalid_expression.txt
PASS  examples/missing_semicolon.txt
PASS  examples/precedence.txt
PASS  examples/undeclared_variable.txt
PASS  examples/valid.txt
10 passed, 0 failed
```

10/10 example programs match their expected output

COVERS

a valid program, precedence, missing ';', undeclared and duplicate variables, two operators in a row, missing and extra ')', division by a variable that is 0, an empty file.

Built with g++ -std=c++17 -Wall -Wextra -pedantic -O2, no warnings.

PROOF

Honest limits, and what comes next

What ErrorBuddy does not do yet, from the original proposal.

KNOWN LIMITS

- **Only int, arithmetic and print.** No if/else, loops or functions.
- **Stops at the first error.** A program with three mistakes needs three runs.
- **No unary minus.** `-5` must be written as `0 - 5`.
- **Values are known at compile time** only because there is no input or loop.
- **TAC is the final output.** There is no optimiser or machine code.

NEXT STEPS

- if / else, while and for loops; comparison and logical operators; printf
- Error recovery: report several errors per run
- Functions and arrays; char and float
- Data-flow analysis once loops and input exist
- Constant folding on TAC, then an assembly back end

CLOSING

Five steps, one story

1	2	3	4	5
Read	**Parse**	**Check**	**Generate**	**Explain**
the lexer tags every token with its line	one function per grammar rule	names and values in the symbol table	three-address code, all or nothing	the line, the rule and one fix

Other compilers ask: "Is this program valid?"
ErrorBuddy also answers: "Where is the mistake, and how do I fix it?"

Vardan Pandya
24BCE2608

Kunal Sharma
24BCE2641

Thank you

github.com/Vardanp1/Compiler-Design-Project

TEST SUITE · make test

Valid

OK | All phases | EXIT 0

PROGRAM

```
1  // A valid ErrorBuddy program
2  int age = 20;
3  int score = 10;
4  score = score + age * 2;
5  int bonus = (score - 5) / 5;
6  print score;
7  print bonus * (age + 1);
```

ACTUAL OUTPUT

Compilation successful

```
age = 20
score = 10
t1 = age * 2
t2 = score + t1
score = t2
t3 = score - 5
t4 = t3 / 5
bonus = t4
print score
t5 = age + 1
t6 = bonus * t5
print t6
```

Matches examples/expected/valid.out

TEST SUITE · make test

Precedence

OK | All phases | EXIT 0

PROGRAM

```
1  int a = 5;
2  int b = 10;
3  int c = a + b * 2;
4  print c;
```

ACTUAL OUTPUT

Compilation successful

```
a = 5
b = 10
t1 = b * 2
t2 = a + t1
c = t2
print c
```

Matches examples/expected/precedence.out

TEST SUITE · make test

Missing semicolon

ERROR | Parser (phase 2) | EXIT 1

PROGRAM

```
1  int a = 5;
2  int b = 10
3  print a + b;
```

ACTUAL OUTPUT

Error on line 2: Expected ';' at the end of the statement.

Suggestion: Add a semicolon after '10'.

Matches examples/expected/missing_semicolon.out

TEST SUITE · make test

Undeclared variable

ERROR | Semantic (phase 3) | EXIT 1

PROGRAM

```
1  int total = 50;
2  total = total + marks;
3  print total;
```

ACTUAL OUTPUT

Error on line 2: Variable 'marks' has not been declared.

Suggestion: Declare it using 'int marks = 0;' before this line.

Matches examples/expected/undeclared_variable.out

TEST SUITE · make test

Duplicate declaration

ERROR | Semantic (phase 3) | EXIT 1

PROGRAM

```
1  int age = 18;
2  int year = 2026;
3  print age;
4  int age = 19;
```

ACTUAL OUTPUT

Error on line 4: Variable 'age' has already been declared (on line 1).

Suggestion: Use a different variable name, or assign a new value with 'age = ...;'.

Matches examples/expected/duplicate_declaration.out

TEST SUITE · make test

Invalid expression

ERROR | Parser (phase 2) | EXIT 1

PROGRAM

```
1  int x = 4;
2  int y = x + * 3;
3  print y;
```

ACTUAL OUTPUT

Error on line 2: Two operators in a row: '+' followed by '*'.

Suggestion: Remove one of them, or put a number or variable between them.

Matches examples/expected/invalid_expression.out

TEST SUITE · make test

Incorrect parentheses

ERROR | Parser (phase 2) | EXIT 1

PROGRAM

```
1  int x = 2;
2  int y = (x + 3 * 4;
3  print y;
```

ACTUAL OUTPUT

Error on line 2: Missing ')' to close the '(' opened on line 2.

Suggestion: Add ')' after '4'.

Matches examples/expected/incorrect_parentheses.out

TEST SUITE · make test

Extra parenthesis

ERROR | Parser (phase 2) | EXIT 1

PROGRAM

```
1  int a = 5;
2  int b = (a + 2)) * 3;
3  print b;
```

ACTUAL OUTPUT

Error on line 2: Found an extra ')' that has no matching '('.

Suggestion: Remove the extra ')' or add a matching '(' earlier.

Matches examples/expected/extra_parenthesis.out

TEST SUITE · make test

Division by zero

ERROR | Semantic (phase 3) | EXIT 1

PROGRAM

```
1  int a = 10;
2  int zero = a - 10;
3  int b = a / zero;
4  print b;
```

ACTUAL OUTPUT

Error on line 3: Division by zero. Variable 'zero' is 0 at this point, so dividing by it is impossible.

Suggestion: Make sure the value on the right of '/' is not zero.

Matches examples/expected/division_by_zero.out

TEST SUITE · make test

Empty

ERROR | Parser (phase 2) | EXIT 1

PROGRAM

```
1  (empty file)
```

ACTUAL OUTPUT

Error on line 1: The program is empty. There is nothing to compile.

Suggestion: Write at least one statement, for example 'int x = 5;'.

Matches examples/expected/empty.out

CASE STUDY · fix belongs on line 2

M01 Missing ; (next line is print)

ERROR | Parser (phase 2) | EXIT 1

PROGRAM

```
1  int a = 5;
2  int b = 10
3  print a + b;
```

ACTUAL OUTPUT

```
Error on line 2: Expected ';' at the end of the statement.
Suggestion: Add a semicolon after '10'.
gcc 11.4: line 3: expected ',' or ';' before 'printf'
```

CASE STUDY · fix belongs on line 2

M02 Missing ; (blank line + comment after)

ERROR | Parser (phase 2) | EXIT 1

PROGRAM

```
1  int a = 5;
2  int b = 10
3
4  // show the total
5  print a + b;
```

ACTUAL OUTPUT

```
Error on line 2: Expected ';' at the end of the statement.
Suggestion: Add a semicolon after '10'.
gcc 11.4: line 5: expected ',' or ';' before 'printf'
```

CASE STUDY · fix belongs on line 2

M03 Missing ; (next line is assignment)

ERROR | Parser (phase 2) | EXIT 1

PROGRAM

```
1  int a = 5;
2  int b = 10
3  b = b + a;
4  print b;
```

ACTUAL OUTPUT

```
Error on line 2: Expected ';' at the end of the statement.
Suggestion: Add a semicolon after '10'.
gcc 11.4: line 3: expected ',' or ';' before 'b'
```

CASE STUDY · fix belongs on line 3

M04 Missing ; on last statement

ERROR | Parser (phase 2) | EXIT 1

PROGRAM

```
1  int a = 5;
2  int b = a * 2;
3  print b
```

ACTUAL OUTPUT

```
Error on line 3: Expected ';' at the end of the statement.
Suggestion: Add a semicolon after 'b'.
gcc 11.4: line 3: expected ';' before 'return'
```

CASE STUDY · fix belongs on line 2

M05 Missing)

ERROR | Parser (phase 2) | EXIT 1

PROGRAM

```
1  int x = 2;
2  int y = (x + 3 * 4;
3  print y;
```

ACTUAL OUTPUT

```
Error on line 2: Missing ')' to close the '(' opened on line 2.
Suggestion: Add ')' after '4'.
gcc 11.4: line 2: expected ')' before ';' token
```

CASE STUDY · fix belongs on line 3

M06 Missing) (expression split over lines)

ERROR | Parser (phase 2) | EXIT 1

PROGRAM

```
1  int x = 2;
2  int y = (x +
3      3 * 4;
4  print y;
```

ACTUAL OUTPUT

```
Error on line 3: Missing ')' to close the '(' opened on line 2.
Suggestion: Add ')' after '4'.
gcc 11.4: line 3: expected ')' before ';' token
```

CASE STUDY · fix belongs on line 2

M07 Extra)

ERROR | Parser (phase 2) | EXIT 1

PROGRAM

```
1  int a = 5;
2  int b = (a + 2)) * 3;
3  print b;
```

ACTUAL OUTPUT

Error on line 2: Found an extra ')' that has no matching '('.

Suggestion: Remove the extra ')' or add a matching '(' earlier.

gcc 11.4: line 2: expected ',' or ';' before ')' token

CASE STUDY · fix belongs on line 2

M08 Two operators in a row

ERROR | Parser (phase 2) | EXIT 1

PROGRAM

```
1  int x = 4;
2  int y = x + * 3;
3  print y;
```

ACTUAL OUTPUT

Error on line 2: Two operators in a row: '+' followed by '*'.

Suggestion: Remove one of them, or put a number or variable between them.

gcc 11.4: line 2: invalid type argument of unary '*' (have 'int')

CASE STUDY · fix belongs on line 2

M09 Operator with no right operand

ERROR | Parser (phase 2) | EXIT 1

PROGRAM

```
1  int x = 4;
2  int y = x + ;
3  print y;
```

ACTUAL OUTPUT

Error on line 2: The operator '+' has no value on its right side.

Suggestion: Add a number or variable after '+', or remove the '+'.

gcc 11.4: line 2: expected expression before ';' token

CASE STUDY · fix belongs on line 3

M10 Missing operator between values

ERROR | Parser (phase 2) | EXIT 1

PROGRAM

```
1  int a = 2;
2  int b = 3;
3  int c = a b;
4  print c;
```

ACTUAL OUTPUT

Error on line 3: Missing operator between 'a' and 'b'.

Suggestion: Put an operator such as '+' or '*' between them, or end the statement with ';'.

gcc 11.4: line 3: expected ',' or ';' before 'b'

CASE STUDY · fix belongs on line 2

M11 Missing value after =

ERROR | Parser (phase 2) | EXIT 1

PROGRAM

```
1  int a = 1;
2  int b = ;
3  print a;
```

ACTUAL OUTPUT

Error on line 2: Missing value after '='.

Suggestion: Write a value after '=', for example 'x = 5;'.

gcc 11.4: line 2: expected expression before ';' token

CASE STUDY · fix belongs on line 2

M12 Empty parentheses

ERROR | Parser (phase 2) | EXIT 1

PROGRAM

```
1  int a = 1;
2  int b = a * ();
3  print b;
```

ACTUAL OUTPUT

Error on line 2: Empty parentheses '()' are not allowed.

Suggestion: Put an expression inside them, for example '(a + 1)'.

gcc 11.4: line 2: expected expression before ')' token

CASE STUDY · fix belongs on line 2

M13 Undeclared variable

ERROR | Semantic (phase 3) | EXIT 1

PROGRAM

```
1  int total = 50;
2  total = total + marks;
3  print total;
```

ACTUAL OUTPUT

```
Error on line 2: Variable 'marks' has not been declared.
Suggestion: Declare it using 'int marks = 0;' before this line.
gcc 11.4: line 2: 'marks' undeclared (first use in this function)
```

CASE STUDY · fix belongs on line 3

M14 Misspelled variable

ERROR | Semantic (phase 3) | EXIT 1

PROGRAM

```
1  int score = 10;
2  int bonus = 5;
3  scroe = score + bonus;
4  print score;
```

ACTUAL OUTPUT

```
Error on line 3: Variable 'scroe' has not been declared.
Suggestion: Did you mean 'score'? If 'scroe' is a new variable,
write 'int scroe = ...;'.
gcc 11.4: line 3: 'scroe' undeclared (first use in this function);
did you mean 'score'?
```

CASE STUDY · fix belongs on line 2

M15 Misspelled print keyword

ERROR | Parser (phase 2) | EXIT 1

PROGRAM

```
1  int x = 3;
2  prnt x;
```

ACTUAL OUTPUT

Error on line 2: Unknown word 'prnt'.

Suggestion: Did you mean the keyword 'print'? Keywords are lowercase and followed by a space.

gcc 11.4: line 2: implicit declaration of function 'prinft'; did you mean 'printf'? [-Wimplicit-function-declaration]

CASE STUDY · fix belongs on line 4

M16 Duplicate declaration

ERROR | Semantic (phase 3) | EXIT 1

PROGRAM

```
1  int age = 18;
2  int year = 2026;
3  print age;
4  int age = 19;
```

ACTUAL OUTPUT

Error on line 4: Variable 'age' has already been declared (on line 1).

Suggestion: Use a different variable name, or assign a new value with 'age = ...;'.

gcc 11.4: line 4: redefinition of 'age'

CASE STUDY · fix belongs on line 1

M17 Variable used in its own declaration

ERROR | Semantic (phase 3) | EXIT 1

PROGRAM

```
1  int a = a + 1;
2  print a;
```

ACTUAL OUTPUT

Error on line 1: Variable 'a' is used in its own declaration, before it has a value.

Suggestion: Give it a starting value that does not use 'a', for example 'int a = 0;'.

gcc 11.4: line 1: 'a' is used uninitialized [-Wuninitialized]

CASE STUDY · fix belongs on line 2

M18 Division by literal 0

ERROR | Semantic (phase 3) | EXIT 1

PROGRAM

```
1  int a = 10;
2  int b = a / 0;
3  print b;
```

ACTUAL OUTPUT

Error on line 2: Division by zero. You are dividing by the number 0.

Suggestion: Make sure the value on the right of '/' is not zero.

gcc 11.4: line 2: division by zero [-Wdiv-by-zero]

CASE STUDY · fix belongs on line 3

M19 Division by a variable that is 0

ERROR | **Semantic (phase 3)** | **EXIT 1**

PROGRAM

```
1  int a = 10;
2  int zero = a - 10;
3  int b = a / zero;
4  print b;
```

ACTUAL OUTPUT

Error on line 3: Division by zero. Variable 'zero' is 0 at this point, so dividing by it is impossible.

Suggestion: Make sure the value on the right of '/' is not zero.

gcc 11.4: nothing reported

CASE STUDY · fix belongs on line 1

M20 Division by an expression that is 0

ERROR | **Semantic (phase 3)** | **EXIT 1**

PROGRAM

```
1  int a = 8 / (4 - 4);
2  print a;
```

ACTUAL OUTPUT

Error on line 1: Division by zero. The expression on the right of '/' works out to 0.

Suggestion: Make sure the value on the right of '/' is not zero.

gcc 11.4: line 1: division by zero [-Wdiv-by-zero]

CASE STUDY · fix belongs on line 1

M21 Arithmetic overflow

ERROR | Semantic (phase 3) | EXIT 1

PROGRAM

```
1  int big = 2000000000 * 2;
2  print big;
```

ACTUAL OUTPUT

Error on line 1: The result of '*' here is too large to fit in an int.

Suggestion: Use smaller numbers. An int must be between -2147483648 and 2147483647.

gcc 11.4: line 1: integer overflow in expression of type 'int' results in '-294967296' [-Woverflow]

CASE STUDY · fix belongs on line 1

M22 Literal too large for int

ERROR | Lexer (phase 1) | EXIT 1

PROGRAM

```
1  int n = 99999999999;
2  print n;
```

ACTUAL OUTPUT

Error on line 1: The number 99999999999 is too large. An int can be at most 2147483647.

Suggestion: Use a smaller number.

gcc 11.4: line 1: overflow in conversion from 'long int' to 'int' changes value from '99999999999' to '1215752191' [-Woverflow]

CASE STUDY · fix belongs on line 2

M23 Unknown symbol

ERROR | Lexer (phase 1) | EXIT 1

PROGRAM

```
1  int a = 5;
2  int b = a @ 2;
3  print b;
```

ACTUAL OUTPUT

Error on line 2: Unknown symbol '@'. ErrorBuddy only understands letters, digits and + - * / = ( ) ;

Suggestion: Remove the symbol or replace it with one of the supported symbols.

gcc 11.4: line 2: stray '@' in program

CASE STUDY · fix belongs on line 1

M24 Name starts with a digit

ERROR | Lexer (phase 1) | EXIT 1

PROGRAM

```
1  int 2nd = 5;
2  print 2nd;
```

ACTUAL OUTPUT

Error on line 1: '2nd' is not a valid name. Variable names cannot start with a digit.

Suggestion: Start the name with a letter, for example 'value1' instead of '1value'.

gcc 11.4: line 1: invalid suffix "nd" on integer constant